SCOUTPATH - APLIKÁCIA PRE SLOVENSKÝ SKAUTING

Bc. Tomáš Sládek

Diplomová práca
Fakulta informačních technologií
České vysoké učení technické v Praze
Katedra softwarového inženýrství
Študijný program: Informatika
Špecializácia: Webové inženýrství
Vedúci: Ing. Josef Pavlíček, Ph.D.
1. apríla 2026

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ ČVUT V PRAZE

Zadání diplomové práce

Název:	ScoutPath - aplikácia pre Slovenský skauting
Student:	Bc. Tomáš Sládek
Vedoucí:	Ing. Josef Pavlíček, Ph.D.
Studijní program:	Informatika
Obor / specializace:	Webové inženýrství
Katedra:	Katedra softwarového inženýrství
Platnost zadání:	do konce letního semestru 2026/2027

Pokyny pro vypracování

Cieľom práce je vytvorenie progresívnej webovej aplikácie na správu rozvoja členov skautského zboru. Aplikácia by mala obsahovať program Slovenského skautingu, primárne programovú zložku stupne napredovania. Aplikácia by mala umožniť synchronizáciu dát používateľov pomocou webových technológií.
- Analyzujte funkčné a nefunkčné požiadavky zákazníka.
- Na základe požiadaviek navrhnite architekturu aplikácie.
- Navrhnite prívetivé používateľské rozhranie.
- Implementujte HI-FI prototyp.
- Aplikáciu otestujte vhodnými testami.

Popíšte dosiahnuté výsledky a navrhnite možný ďalší vývoj.

Elektronicky schválil/a Ing. Jaroslav Kuchař, Ph.D. dne 27. ledna 2026 v Praze.

České vysoké učení technické v Praze
Fakulta informačních technologií
© 2026 Bc. Tomáš Sládek. Všetky práva vyhradené.
Táto práca vznikla ako školské dielo na FIT ČVUT v Prahe. Práca je chránená medzinárodnými predpismi a zmluvami o autorskom práve a právach súvisiacich s autorským právom. Na jej využitie, s výnimkou bezodplatných zákonných licencií a nad rámec oprávnení uvedených vo Vyhlásenie, je nutný súhlas autora.

Odkaz na túto prácu: Sládek Tomáš. *ScoutPath - aplikácia pre Slovenský skauting.* Diplomová práca. České vysoké učení technické v Praze, Fakulta informačních technologií, 2026.

Chtěl bych poděkovat především sit amet, consectetuer adipiscing elit. Curabitur sagittis hendrerit ante. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Cras pede libero, dapibus nec, pretium sit amet, tempor quis. Sed vel lectus. Donec odio tempus molestie, porttitor ut, iaculis quis, sem. Suspendisse sagittis ultrices augue.

Vyhlásenie

Prohlašuji, že jsem předloženou práci vypracoval samostatně a že jsem uvedl veškeré použitéinformační zdroje v souladu s Metodickým pokynem o dodržování etických principů při přípravěvysokoškolských závěrečných prací.

Beru na vědomí, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., autorského zákona, ve znění pozdějších předpisů. V souladu s ust. § 2373 odst. 2 zákona č. 89/2012 Sb., občanský zákoník, ve znění pozdějších předpisů, tímto uděluji nevýhradní oprávnění (licenci) k užití této mojí práce, a to včetně všech počítačových programů, jež jsou její součástí či přílohou a veškeré jejich dokumentace (dále souhrnně jen „Dílo“), a to všem osobám, které si přejí Dílo užít. Tyto osoby jsou oprávněny Dílo užít jakýmkoli způsobem, který nesnižuje hodnotu Díla a za jakýmkoli účelem (včetně užití k výdělečným účelům). Toto oprávnění je časově, teritoriálně i množstevně neomezené.

Prohlašuji, že jsem v průběhu příprav a psaní závěrečné práce použil/-a nástroje umělé inteligence. Vygenerovaný obsah jsem ověřil/-a. Stvrzuji, že jsem si vědom/-a, že za obsah závěrečné práce plně zodpovídám.

V Prahe dňa 1. apríla 2026

Abstrakt

Fill in the abstract of this thesis in Czech. Lorem ipsum dolor sit amet. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Cras pede libero, dapibus nec, pretium sit amet, tempor quis. Sed vel lectus. Donec odio tempus molestie, porttitor ut, iaculis quis, sem. Suspendisse sagittis ultrices augue.

Kľúčové slová enter, comma, separated, list, of, keywords, in, CZECH

Abstract

Fill in the abstract of this thesis in English. Lorem ipsum dolor sit amet. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Cras pede libero, dapibus nec, pretium sit amet, tempor quis. Sed vel lectus. Donec odio tempus molestie, porttitor ut, iaculis quis, sem. Suspendisse sagittis ultrices augue.

Keywords enter, comma, separated, list, of, keywords, in, ENGLISH

Obsah

1 Teoretický základ **3**
1.1 Slovenský skauting . . . 3
1.2 Štruktúra Slovenského skautingu . . . 3
1.3 Program SLSK . . . 4
1.3.1 Stupne Napredovania . . . 4
1.3.1.1 Skautský chodník . . . 5
1.3.1.2 Prvý skaut . . . 5
1.3.1.3 Neznáme cesty . . . 6
1.3.1.4 Posledný vrchol . . . 6
1.3.2 Rangerský horizont . . . 6
1.3.3 Odborky . . . 6
1.3.4 Výzvy . . . 7
1.3.5 Voľné programové moduly . . . 7
1.3.6 Najvyššie programové ocenenia . . . 7
1.4 Analýza podobných riešení . . . 7
1.4.1 Path To Eagle . . . 8
1.4.2 Scout Champ . . . 8
1.5 Softvérové inžinierstvo . . . 8
1.5.1 Životný cyklus vývoja softvéru SDLC . . . 8
1.5.2 Zber požiadaviek . . . 10
1.5.2.1 Kategorizácia požiadaviek . . . 11
1.6 User centric design . . . 11
1.7 Tvorba používateľského rozhrania . . . 11
1.7.1 Základné prvky používateľského rozhrania . . . 11
1.7.2 Najlepšie praktiky pre tvorbu UI do prírody . . . 11
1.7.3 Prototyp . . . 12
1.7.4 Persóny . . . 12
1.8 Testovanie používateľského rozhrania . . . 13
1.8.1 Testovania mladistvým . . . 13
1.9 Webové inžinierstvo . . . 13
1.9.1 Synchronizácia dát . . . 13
1.9.2 Klient-server architektúra . . . 14
1.9.3 Peer-to-peer architektúra . . . 14
1.9.4 API . . . 14
1.9.4.1 Druhy API . . . 15
1.9.5 OAuth . . . 15

1.9.6 Cloud computing 15
1.9.6.1 Typy služieb 15
1.9.7 Backend ako služba 16
1.9.8 Webscraping 16
1.9.8.1 Fázy procesu webscraping 16
1.9.8.2 Etické otázky ohľadom procesu webscraping . 17

2 Analýza **18**
2.1 Prípady použitia 18
2.2 Zber požiadaviek 19
2.2.1 Požiadavky 19
2.3 Cieľová skupina aplikácie 20
2.4 Potreby backendu 20
2.5 Ne-cloudové riešenie 21
2.6 Cloudové riešenie backendu 21
2.6.1 IaaS riešenia 21
2.6.1.1 AWS 21
2.6.1.2 OCI 21
2.6.1.3 21
2.6.2 PaaS riešenia 21
2.6.3 BaaS riešenia 21
2.6.4 FireBase 21
2.6.4.1 21

3 Návrh **22**
3.1 Doménový model 22
3.2 Procesy 22
3.3 Aktualizácie obsahu programu SLSK 22
3.4 Architektúra aplikácie 22
3.5 Voľba programovacieho jazyku 22
3.6 LoFi prototyp 22
3.7 Testovanie LoFi prototypu 22
3.8 HiFi prototyp 22
3.9 Testovanie HiFi prototypu 23
3.10 API štruktúra 23

4 Implementácia **24**
4.1 Získavanie dát 24
4.2 Flutter 24
4.3 Nasadenie aplikácie 24

5 Testovanie **25**
5.1 Používateľské testovanie 25
5.2 Heuristická analýza 25
5.3 Automatizované testovanie 25

6 Záver **26**
6.1 Budúci vývoj 26

A Nějaká příloha **27**

Obsah příloh **30**

Zoznam obrázkov

Zoznam tabuliek

Zoznam výpisov kódu

Zoznam skratiek

SLSK Slovenský skauting

Úvod

Skauting predstavuje medzinárodné mládežnícke hnutie zamerané na rozvoj osobnosti prostredníctvom praktických aktivít v prírode, spolupráce v kolektíve a neformálneho vzdelávania. Na týchto princípoch funguje dobrovoľnícka organizácia Slovenský skauting, ktorá na Slovensku zabezpečuje programové aktivity pre deti a mládež.

Program organizácie je realizovaný najmä prostredníctvom pravidelných stretnutí družín, ktoré predstavujú základnú organizačnú jednotku skautingu. Družina je spravidla tvorená skupinou približne dvanástich členov vedených radcom. Radca je zodpovedný za prípravu programu stretnutí a koordináciu aktivít družiny.

Pri plánovaní programu však radcovia často čelia náročnej úlohe zosúladiť spoločné aktivity družiny s individuálnym napredovaním jednotlivých členov v rámci skautského programu.

Napriek existencii programovej štruktúry organizácie Slovenský skauting môžu radcovia a vedúci pri plánovaní aktivít a sledovaní pokroku členov narážať na viacero praktických problémov. Programová ponuka je síce dostupná prostredníctvom oficiálnych webových stránok organizácie a tlačených príručiek, ktoré obsahujú informácie o stupňoch napredovania, odborkách a ďalších programových prvkoch, ich používanie pri každodennej práci s družinou však je často zdĺhavé a nepraktické.

Zároveň existuje veľké množstvo digitálnych nástrojov využívaných na komunikáciu, organizáciu aktivít alebo evidenciu úloh. Používanie viacerých oddelených nástrojov však môže viesť k roztriešteniu informácií a zvyšovať organizačnú záťaž vedúcich a radcov. Táto situácia vytvára priestor pre komplexné riešenie, ktoré by integrovalo prehľadné sprístupnenie programovej ponuky s jednoduchým nástrojom na sledovanie pokroku členov a podporu organizácie aktivít družiny.

Cieľ práce

Cieľom tejto diplomovej práce je návrh, implementácia a otestovanie aplikácie určenej pre členov organizácie Slovenský skauting. Aplikácia má prehľadnejšie sprístupniť programovú ponuku organizácie jej členom a znížiť organizačnú záťaž pri sledovaní individuálneho pokroku členov zo strany radcov a vedúcich.

Aby aplikácia naplnila svoj účel, musí byť intuitívna pre široké vekové spektrum používateľov. Súčasťou práce je preto dôkladný návrh používateľského rozhrania (UI/UX), samotná implementácia systému, ktorý zjednotí zobrazovanie programových prvkov so správou družinovej agendy, a následné používateľské testovanie vytvoreného riešenia.

Práca je rozdelená do šiestich hlavných kapitol. Prvá kapitola sa venuje teoretickému základu, kde definujeme princípy organizácie Slovenský skauting, analyzujeme existujúce digitálne riešenia a popisujeme technologické východiská z oblasti softvérového inžinierstva a tvorby používateľských rozhraní. Druhá kapitola obsahuje analýzu cieľovej skupiny a definovanie funkčných aj nefunkčných požiadaviek na aplikáciu. V tretej kapitole popisujeme proces návrhu, od doménového modelu a architektúry až po tvorbu a testovanie prototypov nízkej a vysokej vernosti. Štvrtá kapitola je zameraná na samotnú implementáciu v prostredí Flutter a proces získavania dát. Piata kapitola vyhodnocuje úspešnosť riešenia prostredníctvom používateľského testovania, heuristickej analýzy a automatizovaných testov. V závere sumarizujeme dosiahnuté výsledky a navrhujeme možnosti pre budúci vývoj aplikácie.

Kapitola 1
Teoretický základ

Táto kapitola sa venuje predstaveniu prostredia Slovenského skautingu. Následne opisuje aplikácie zaoberajúce sa podobnou tématikou v prostredí iných krajín. Ďalej sú predstavené moderné technolólgie a metódy ktoré sú používané na vývoj aplikácií.

1.1 Slovenský skauting

Slovenský skauting (ďalej len SLSK) je dobrovoľná, nezávislá, nepolitická, nezisková, výchovná organizácia mladých ľudí, prístupná všetkým bez rozdielu pohlavia, pôvodu, národnosti, rasy alebo náboženstva, ktorí chcú žiť a konať v súlade s poslaním, princípmi a výchovnou metódou definovanou zakladateľom hnutia Robertom Baden-Powellom. SLSK pôsobí najmä v oblasti neformálneho vzdelávania detí a mládeže, vzdelávania dobrovoľných pracovníkov, dospelých a mladých lídrov. Medzi kľúčové piliere činnosti patrí podpora občianskej participácie výchova k demokratickým hodnotám, prevencia závislostí, ochrana životného prostredia, realizácia sociálnych aktivít a práce so znevýhodnenými skupinami. Organizácia taktiež aktívne buduje kultúru dobrovoľníctva a zmysluplného trávenia voľného času. [1]

1.2 Štruktúra Slovenského skautingu

Organizačná štruktúra SLSK je postavená na hierarchickom systéme, ktorý zabezpečuje stabilitu a metodickú podporu na všetkých úrovniach. Najvyšším orgánom je Skautský snem, ktorý sa schádza každé tri roky a volí Náčelníctvo zodpovedné za riadenie organizácie. Regionálnu úroveň tvoria oblasti a zbory, ktoré poskytujú právnu a finančnú podporu nižším organizačným zložkám – oddielom.

Z hľadiska priamej výchovnej činnosti je však kľúčovou jednotkou družina. Predstavuje základnú bunku hnutia, ktorú tvorí malý kolektív spravidla piatich

až ôsmich detí. Družinu vedie radca – starší a skúsený člen, ktorý pripravuje program pravidelných týždenných stretnutí. Práve v tomto malom kolektíve dochádza k praktickej realizácii skautskej výchovnej metódy prostredníctvom hier, príbehov a rozvoja zručností. [2]

1.3 Program SLSK

Skautský program predstavuje ucelený systém výchovy a vzdelávania, ktorý sa zameriava na štyri základné oblasti, definované ako piliere rozvoja. Prvým z nich je charakter, ktorý je rozvíjaný najmä prostredníctvom družinového systému, hodnôt ukotvených v skautskom zákone a prijímaním osobnej zodpovednosti v kolektíve. Oblasť sily a zdravia je budovaná fyzickými aktivitami, hrami v prírode a dôrazom na zdravú výživou, zatiaľ čo zručnosti a schopnosti členovia nadobúdajú praktickou činnosťou na táboroch a výpravách, kde je kľúčovým prvkom metodika učenia sa praxou. Posledným pilierom je služba druhým, zakorenená vo všímavosti voči okoliu a ochote pomáhať, čím skauting formuje mravných a prosociálnych ľudí.

Cieľom tohto snaženia je komplexný rozvoj jedinca v dimenziách intelektu, charakteru, emocionálnosti a duchovnosti. Hoci sa skautské aktivity môžu laickej verejnosti javiť ako voľnočasové, v skutočnosti predstavujú pridanú hodnotu neformálnej výchovy, ktorá formuje človeka na celý život. Na konci tohto procesu stojí jedinec, ktorý si uvedomuje svoje kvality, rešpektuje potreby svojho okolia a koná v súlade s princípmi úcty a rešpektu. [2]

Na praktickú realizáciu týchto cieľov využíva Slovenský skauting konkrétne programové prvky, ktoré tvoria metodické jadro organizácie. Ide predovšetkým o príručky osobného napredovania, ktoré slúžia ako základný nástroj na sledovanie individuálneho rastu člena. Tento systém dopĺňajú odborky, zamerané na rozvoj konkrétnych talentov, a výzvy, ktoré preverujú pevnú vôľu a vytrvalosť jednotlivca. Ponuku uzatvárajú voľné programové moduly, teda tematické bloky aktivít reagujúce na aktuálne spoločenské témy. [1]

1.3.1 Stupne Napredovania

Stupne napredovania tvoria metodický základ programu Slovenského skautingu a slúžia ako nástroj na stimuláciu osobného rastu. Sú odstupňované podľa vekových kategórií, čím reflektujú psychomotorický a sociálny vývin členov od predškolského veku až po mladú dospelosť:

- **4 – 6 rokov:** Janík a Grétka (predškoláci),
- **7 – 10 rokov:** Vlčia stopa (vĺčatá a včielky),
- **11 – 14 rokov:** Skautský chodník (skauti a skautky),
- **15 – 18 rokov:** Rangerský horizont (rangeri a rangerky),

- **19 – 24 rokov:** Roverský svet (roveri a roverky).

Špecifické postavenie má stupeň Nováčik, ktorý je určený pre nových členov starších ako 11 rokov a slúži na základnú orientáciu v organizácii a jej hodnotách. [3]

1.3.1.1 Skautský chodník

Skautský chodník je kľúčovým programovým prvkom pre skautskú vekovú kategóriu. Je rozdelený na tri postupné etapy: Prvý skaut, Neznáme cesty a Posledný vrchol. Každá etapa je koncipovaná na obdobie približne jedného školského roka.

Program sa zameriava na všestranný rozvoj, ktorý je pre lepšiu prehľadnosť v príručkách rozdelený do štyroch tematických kategórií, odlíšených farebným kódovaním:

- **Charakter (modrá):** zameriava sa na rozvoj vnútornej integrity, hodnôt a sociálnych vzťahov v rámci družiny.
- **Služba (červená):** zahŕňa ekológiu, spoznávanie kultúr, komunikačné schopnosti a duchovné dedičstvo skautingu.
- **Zdravie a sila (žltá):** venuje sa fyzickej zdatnosti, hygiene a prežitiu v prírode.
- **Schopnosti a zručnosti (zelená):** orientuje sa na intelektuálny rozvoj a praktické zručnosti, ako je varenie či manuálna tvorba.

Každá kategória obsahuje kapitoly so sadou úloh. Metodika SLSK umožňuje radcovi alebo vedúcemu upraviť náročnosť úlohy tak, aby zodpovedala individuálnym potrebám člena, pričom sa musí zachovať jej pôvodná téma. Splnenie úloh sa overuje tromi spôsobmi v závislosti od ich povahy: potvrdením radcom alebo vedúcim, potvrdením od rodiča alebo samooverením, ktoré stavia na skautskej česti člena. Po splnení definovaného množstva úloh v kategórii sa skautovi odomyká možnosť plniť záverečnú výzvu.[4, 5, 6]

1.3.1.2 Prvý skaut

Predstavuje úvodnú etapu chodníka (vek 11 – 12 rokov), ktorej symbolickým sprievodcom je zakladateľ hnutia Robert Baden-Powell. Z hľadiska programovej štruktúry je táto etapa najviac fixná; úlohy sú vopred definované bez možnosti voľby, čo zodpovedá potrebe jasného smerovania v úvode skautského veku. Výzvy sa sprístupňujú až po splnení všetkých úloh v danej kategórii. [4]

1.3.1.3 Neznáme cesty

Druhá etapa (vek 12 – 13 rokov) nadväzuje na získané základy a do programu vnáša prvok voliteľnosti. Symbolický rámec prenáša skauta do štyroch krajín (ľad, lúka, púšť, prales), ktoré reprezentujú jednotlivé kategórie rozvoja. V tejto fáze sa zavádza bodový systém: každá kapitola obsahuje jednu povinnú úlohu a súbor voliteľných úloh s rôznou bodovou hodnotou. Cieľom člena je získať potrebný počet bodov pre každú kategóriu. [5]

1.3.1.4 Posledný vrchol

Záverečná etapa (vek 13 – 14 rokov) symbolizuje výstup na vrchol a prípravu na prechod k rangerom. Ponúka najvyššiu mieru slobody vo výbere programu. Člen si volí nielen konkrétne úlohy, ale aj celé kapitoly, ktoré ho zaujímajú. Na rozdiel od predchádzajúcich etáp sa body sčítavajú kumulatívne naprieč všetkými kategóriami, čo umožňuje skautovi profilovať sa v oblastiach, ktoré sú mu najbližšie. [6]

1.3.2 Rangerský horizont

Rangerský horizont predstavuje programovú etapu určenú pre vekovú kategóriu 15 – 18 rokov a je koncipovaný na obdobie maximálne troch rokov. Programová štruktúra sa v tejto fáze odkláňa od fixných kategórií Skautského chodníka a sústreďuje sa na päť kľúčových míľnikov: Osobný rast I, Osobný rast II, Expedícia I, Expedícia II a Rangerský projekt.

Časti Osobný rast I a II obsahujú sériu úloh podobnú tým zo Skautského chodníka, avšak s vyššou mierou náročnosti a dôrazom na sebareflexiu a vedenie príkladom. Expedície sú zamerané na spoznávanie okolia a rozvoj samostatnosti v neznámom prostredí. Na ich splnenie si rangeri volia konkrétny typ výpravy, napríklad putovnú, cyklistickú alebo vodácku expedíciu, pričom druhá expedícia (II) spravidla vyžaduje vyššiu mieru plánovania a náročnejší terén či dĺžku trvania. Rangerský projekt je vyvrcholením tejto etapy a je určený na tímové riešenie spoločenskej, ekologickej alebo skautskej problematiky, ktorú si rangeri sami zvolia a nechajú schváliť oddielovým vodcom. Celý program Rangerského horizontu je navyše navrhnutý tak, aby bol kompatibilný s Medzinárodnou cenou vojvodu z Edinburghu (DofE), čo umožňuje členom paralelné uznávanie ich aktivít v oboch programoch.[7]

1.3.3 Odborky

Odborky predstavujú špecializovanú časť skautského programu, ktorá funguje komplementárne a nezávisle od stupňov napredovania. Ich hlavným cieľom je umožniť členom hlbšie sa orientovať v konkrétnej oblasti záujmu, rozvíjať špecifické talenty a v staršom veku pomáhať pri voľbe budúceho povolania. Každá

odborka pozostáva zo súboru úloh, ktorých splnenie vedie k nadobudnutiu odbornosti v danej téme (napr. astronóm, zdravotník, informatik či tlmočník).

Metodika SLSK rozlišuje odborky podľa náročnosti a vekového určenia. Pre kategóriu skautov a rangerov sú odborky spravidla štruktúrované do dvoch stupňov:

- **Zelený stupeň:** predstavuje základnú úroveň vedomostí a zručností v danej oblasti.
- **Červený stupeň:** vyžaduje pokročilé znalosti a často nadväzuje na úspešné absolvovanie zeleného stupňa.

1.3.4 Výzvy

Výzvy tvoria špecifickú súčasť programu zameranú na posúvanie hraníc komfortnej zóny a budovanie pozitívnych návykov. Na rozdiel od odboriek, ktoré sú orientované na získavanie odborných znalostí, sa výzvy sústreďujú na charakterové vlastnosti, pevnú vôľu a vytrvalosť. Ich cieľom je postaviť člena do neštandardných situácií, ktoré si vyžadujú plánovanie, disciplínu a schopnosť nevzdať sa pri prvotnom neúspechu. Programová ponuka obsahuje široké spektrum výziev prispôsobených nárokom jednotlivých vekových kategórií.[8]

1.3.5 Voľné programové moduly

Voľné programové moduly predstavujú voliteľný doplnok skautského programu, ktorý umožňuje členom rozvíjať sa v špecifických oblastiach presahujúcich štandardné stupne napredovania. Každý modul má vlastnú unikátnu metodiku, systém úloh a spôsob overovania.[9]

1.3.6 Najvyššie programové ocenenia

Najvyššie programové ocenenia predstavujú vrcholné méty, ktoré môže člen v rámci jednotlivých vekových kategórií dosiahnuť. Tieto ocenenia nie sú samostatnými súbormi úloh, ale fungujú ako integrovaný súhrn prerekvizít naprieč celou programovou ponukou. Ich získanie je podmienené kombináciou úspešného absolvovania stupňov napredovania, získania definovaného počtu odboriek, splnenia konkrétnych výziev a zloženia skautského sľubu.[10]

1.4 Analýza podobných riešení

Celosvetovosť skautského hnutia dáva priestor na vznik viacerým organizáciam zaoberajúcich sa podobnou tématikou. Hľadanie už existujúcich riešení som realizoval vyhľadávaním na Google Play pre operačný systém android a na App Store pre iOS. Vyhľadávanie bolo pomocou kľúčových slov Scout Výsledkom

Jednou z najväčších organizácií v rámci hnutia je Boy Scouts of America. BSA obsahuje podobnú základnú ponuku ako SLSK v oblasti odboriek a najvyších programových ocenení. Na zobrazenie tejto ponuky existuje viacero aplikácií medzi inými Path to Eagle, ScoutChamp.

1.4.1 Path To Eagle

Path to Eagle je aplikácia na operačný systém iOS určená pre Scouting America. Aplikácia slúží na trackovanie progresu jednotlivých skautov v napredovaní cez hodnosti v rámci programu Scouting America. Aplikácia funguje čisto v offline prostredí a jediná možnosť na synchronizáciu postupu skautov je export progresu na jednom zariadení a import na druhom zariadení. Aplikácia umožňuje vytvorenie viacerých skautov a pre každého sledovať postup samostatne. Ku každému stupňnu napredovania v rámci programu Scoutig America poskytuje pre skauta checklist úloh ktoré musí splniť a umožňuje mu písať si k tomu vlastné poznámky. Ďalej aplikácia obsahuje zoznam odboriek ktoré môže skaut plniť. Ku každej odborke je následne dostupné pdf ktoré je výstrihom z knihy pre program Scouting America.

1.4.2 Scout Champ

Aplikácia Scout Champ má podobný charakter ako aplikácia Path To Eagle. Zhodné sú v cieľovom operačnom systéme a v obsahu programu ktoré ponúkajú. Okrem sledovania postupu jednotlivých členov však umožňuje aj zdieľanie progresu cez cloud. Progres však zdieľa spôsobo ktorým ho môže ktokoľvek s linkom prepísať bez žiadneho auditu alebo schvaľovania. Oproti aplikácii Path To Eagle sú odborky pretransformované taktiež do formy checklistu.

1.5 Softvérové inžinierstvo

1.5.1 Životný cyklus vývoja softvéru SDLC

Výber správnej metodológie pre SDLC kladie dôležitú rolu na úspech projektu. SDLC metodológie ponúkajú kostru ktorá sprevádza projekt naprieč návrhom, vývojom a implementáciou softvérového riešenia. SDLC je proces ktorý načŕta fázy a aktivity zapojené do tvorby softvéru od kenceptu cez development po údržbu. Medzi typické fázy SDLC patrí:

Plánovanie definuje rozsah a požiadavky projektu, indetifikuje kľúčových členov a ich role a vytvára plán projektu.

Analýza získava a okumentuje podrobné požiadavky od používateľov a kľúčových členov. Analyzuje zozbierané potreby na pochopenie funkčných a nefunkčných požiadavkov systému.

Dizajn vytvára architektúru aplikácie, definujúc vzájomné interakcie komponent. Navrhuje podobu používateľského rozhrania a používateľských skúsenosti.

Implementácia obsahuje vlastnú tvorbu kódu podľa výstupov z predchádzajúcich fáz napr., požiadavky, návrh UI a ďalších.

Testovanie prebieha viacerých úrovniach od testovania jednotlivých častí až po systémové testy, ktoré testujú systém ako celok. V rámci testovania sa verifikuje splnenie požiadaviek.

Nasadenie prebieha primárne v dvoch fázach, nasadenie do testovacieho prostredia a do ostrého prostredia. V rámci nasadenia sa rieši finálne prostredie v ktorom budú softvér využívať používatelia.

Údržba a podpora sa zameriava na monitorovanie a údržbu softvéru v produkčnom prostredí. Prebiehajú v nej opravy na chyby ktoré sa nenašli v rámci predchádzajúcich fáz. Prípadne sem môžu patriť vylepšenia na základe spätnej väzby od používateľov.

Metodológie SDLC sa líšia primárne v prístupe k týmto fázam. Medzi najnámejšie patrí vodopádová metóda, špirálová metóda, iteratívna metóda, inkrementálna metóda, prototypová metóda, metóda v tvare V, metóda rýchlej aplikácie a agilné metódy.

Vodopád je typický lineárnym priechodom naprieč fázami, kde každá fáza začína po skončení predchádzajúcej. Tento prístup je vhodný na projekty s dobre definovanými požiadavkami na začiatku projektu, a prápadné zmeny sú len minimálné. Pevná štruktúra tejto metódy sa ťažko prispôsobuje zmenám po skončení fáze. Výhodou tejto metódy je dobrá odhadnuteľnosť prostriedkov a času na projekt.

Špiralová metóda rozdeľuje projekt do série iterativných cyklov. V každom cykle sa postupne prejde cez všetky fáze. Na konci cyklu sa spraví vyhodnotenie a spolu so spätnou väzbou vstupujú do plánovania ďalšieho cyklu. Toto umožňuje vyššiu flexibiltu na zmeny. Hlavnou vlastnosťou špirálového modelu je jeho zemranie sa na analýzu rizík. Nevýhodou špirálového modelu je jeho komplexnosť a náročnosť na prostriedky. Zároveň mu chýbajú dobre stavené ciele oproti vodopádovej metóde.

Itertívna metóda sa zameriava na postupný vývoj a zdokonalenie softvéru pomocou iterativných cyklov. Umožňuje kontinuálne zlepšovanie a adaptáciu na zmenu požiadavkov. Výhodou je vytvorenie čiastočného no funkčného produktu na konci každej iterácie. Nevýhodou však môže byť prerastanie rozsahu projektu počas vývojového procesu čo môže viesť k prekročeniu rozpočtu a neskorým časom dodania.

Inkrementálna metóda rozdelí na začiatku projekt na ucelené celky. Tieto celky sú následne vyvíjané nezávisle a inrementy sú sekvenčne integrované do projektu. Hlavnou výhodou je možnosť paralalelizmu, no toto zároveň prináša

väčsie nároky na koordinovanie tímu. Táto metóda je vhodná len na projekty ktoré umožňujú rozdelenie na nezávislé časti.

Prototypová metóda je charakterizovaná čo najrýchlejším vytvorením prototypuna získanie spätnej väzby od používateľov. Prototypová metóda obsahuje takisto viacero iterácií, kde spätná väzba z predchádzajúcej iterácie pomáha dotvárať požiadavky na ďalšiu iteráciu. Pravidelná spätná väzba prispieba k zlepšeniu komunikácie medzi vývojovým tímom a koncovými používateľmi. Medzi nevýhody patrí náročný odhad rozpočtu a dĺžky práce, s neznámim počtom iterácií vopred. Zároveň môžu používatelia nadobudnúť pocit že sa jedná o hotový systém aj keď v realite ide iba o prototyp.

Metóda v tvare V je nadstavbou nad klasickou vodopádovou metódou. Oproti vodopádovej metóde obsahuje proces verifikácie a validácie ktorý je aktívny paralelne s hlavnými fázami vodopádovej metódy. Každá vývojová fáza má prislúchajúcu testovaciu fázu a až po skončení oboch sa prechádza do ďalšej fáze. Takto nastavený proces znižuje riziko vynechania nejakej veci pred pokračovaním vo vývoji.

Metóda rýchlej aplikácie je druh inkrementálnej metódy so zameraním sa na čo najrýchlejšie vytvorenie aplikácie. Často sú používané nástroje na tvorbu kódu a vlastný kód je držaný na minime. Zároveň sa zameriava na modularizáciu softvéru a paralelný vývoj nezávislých modulov čo rapídne znižuje dobu vývoja. Táto metóda je vhodná pre väčšie tími kde je dostatok vývojarov na paralelný vývoj softvéru.

Agilná metodológia reprezentuje rodinu iteratívnych a inkrementálnych metód ktoré prioritizujú adaptabilitu a sploprácu so zákazníkom. V základe rozdeľuje projekt na šprinty. Jeden šprint trvá typicky 1 až 4 týždne a predchádza mu scrum, čo je stretnutie na synchronizáciu vykonávanej práce. Agilný prístup zdieľa väčšinu výhod a nevýhod z ostatných iteratívnych metód.[11]

1.5.2 Zber požiadaviek

Softvérové požiadavky predstavujú spôsob akým môžme zachytiť potreby zákazníka. Upresňujú nám rozsah projektu ako do funkcionilty ktorú zákazník očakáva, tak aj do formy akou budú tie funkcionality využité. Zároveň nám môžu upresňovať rozsah aj z opačnej strany v zmysle funkcionalít a vlastností ktoré softvér obsahovať nebude. Dotvára to celkový obraz zákazníkovi a predchádza sa pomocou toho nedorozumeniam.

Dôležitou úlohou na začiatku projeku je získavanie a dokumentácia požiadaviek. Získavanie požiadaviek je proces ktorý prebieha v kooperácií so zákazníkom, kľúčovými členmi a používateľmi. Existuje viacero spôsobom ktorými sme schopní zozbierať tieto požiadavky.

Rozhovory jeden na jednoho

Skupinové rozhovory

Brainstorming

Cieľová skupina

Dotazník
workshop požiadaviek
Sledovanie používateľa
Analýza rozhrania
Analýza dokumentov
Spätné inžinierstvo
Vytváranie prototypov

1.5.2.1 Kategorizácia požiadaviek

Po zozbieraní požiadviek, prichádza na rad rozdelenie požiadaviek podľa rôznych metrík. Hlavnými metrikami zvyknú bývať dôležitosť požiadavky, na ktorú sa používa kategorizácia MoSCoW. Táto kategorizácia delí požiadavky na tie ktoré musia byť zapracované do softvéru, teda must have, tie ktoré by mali byť zapracované do systému, teda should have, tie ktoré by mohli byť vo výslednom softvéri, teda could have a do poslednej kategorizácie spadajú tie ktoré nebudú implementované do softvéru, teda won't have.

Ďalšou metrikou je FURPS+, ktoré delí požiadavky na funkčné, používateľné, spoľahlivé, výkonostné a

1.6 User centric design

1.7 Tvorba používateľského rozhrania

1.7.1 Základné prvky používateľského rozhrania

Štvrtou heuristikou Jakoba Nielsna je udržanie konzistencie a štandardy pri návrhu používateľského rozhrania. Z tohto dôvodu ustálili základné prvky, ktoré sa používajú v rámci používateľského rozhrania.

1.7.2 Najlepšie praktiky pre tvorbu UI do prírody

Pri tvorbe UI pre aplikácie ktoré budú používané v prírode treba myslieť na prírodné faktory ktoré ovplyvňujú zážitok z používania mobilného zariadenia. Medzi hlavné rozdiely medzi používaním mobilného zariadenia v vnútorných a vonkajších priestoroch je rozdiel v intenzite slnečného žiarenia. Následkom toho treba zvážiť väčší font a väčší kontrast, aby bol obsah dobre viditeľný aj za zhoršenej viditeľnosti. Ďalším veľkým rozdielom je horší prístup k internetu v odľahlých miestach v prírode. Aplikácia ktorá potrebnuje pripojenie na internet na všetky svoje funkcie sa tak stáva nepoužiteľnou bez signálu, prípadne mobilných dát v prírode.[12]

1.7.3 Prototyp

Prototyp slúži na otestovanie použiteľnosti používateľského rozhrania pred tým ako investujeme prostriedky do finálneho riešenia. Podľa vernosti s ktorou sa podobajú na finálny produkt rozlišujeme dve kategórie prototypov. Vernosť môže byť v rôznych oblastiach interaktivita, vizuály alebo obsah a príkazy. Existujú dva druhy prototypov podľa množstva prostriedkov potrebných na ich zostavenie.

Prototyp s nízkou vernosťou umožňuje jednoduché úpravy vďaka jeho jednoduchosti. Jednoduchosť prototypu umožňuje predstavenie základnej štruktúry používateľského rozhrania. Na druhú stranu tento prototyp neposkytuje žiadne takmer žiadnu interaktivitu pre používateľa a všetky prechody sa dejú počas testovania manuálne.

Prototyp s vysokou vernosťou poskytuje reálnejšie odozvy systému. Pri testovaní prototypu s vysokou vernosťou môže nastať nedorozumenie, keď sa môže výsledný systém zdať pripravený na použitie aj keď sa jedná len o prototyp.

1.7.4 Persóny

Persóny sú archetypy, ktoré popisujú rôzne ciele a pozorované vzorce správania používateľov. Každá persóna v sebe zapuzdruje kritické behaviorálne dáta spôsobom, ktorý je zrozumiteľný a zapamätateľný pre dizajnérov aj zainteresované strany. Využívajú formu príbehu, čím zapájajú sociálne a emocionálne aspekty, čo návrhárom pomáha vizualizovať optimálne správanie produktu alebo pochopiť, prečo je navrhovaný dizajn úspešný.

Persóna je fiktívna postava s vlastnou osobnosťou, potrebami a názormi, pričom jej tvorba sa opiera o nasledujúce princípy:

- Je vytvorená na základe dát zozbieraných počas rozhovorov s používateľmi alebo pomocou iných štatistických metód.
- Vzniká implementáciou spoločných vzorcov správania viacerých reálnych používateľov do jednej fiktívnej osoby.
- Dodáva inak abstraktným dátam ľudskú tvár, čo umožňuje tímu lepšie pochopiť potreby koncových používateľov.

V rámci metodiky návrhu sa odporúča rozlišovať tri základné varianty persón, ktoré určujú prioritu implementácie funkcií:

Persóna A (Typický používateľ): Predstavuje primárneho používateľa, ktorý bude využívať všetky funkcionality systému. Používateľské rozhranie musí byť navrhnuté primárne pre ciele tejto osoby a z nich sa následne odvodzujú kľúčové prípady použitia.

Persóna B (Príležitostný používateľ): Systém nie je primárne určený pre túto osobu, ale rozhranie by malo umožňovať naplnenie jej cieľov. Hoci tieto prípady použitia môžu byť pre používateľa časovo alebo energeticky náročnejšie, systém mu musí poskytnúť spôsob, ako ich dokončiť.

Persóna C (Negatívna persóna): Niekedy nazývaná aj antipersóna. Ide o používateľa, ktorý systém nikdy nevyužíva. Do procesu návrhu sa zavádza kvôli kontrastu alebo na definovanie funkcionalít, ktoré by v systéme nemali byť prítomné. Pomáha tímu vyhnúť sa nesprávnemu smerovaniu pri návrhu UI. [13]

1.8 Testovanie používateľského rozhrania

Testovanie použiteľnosti slúži na ohodnotenie kvality dizajnu používateľského rozhrania. Hlavné kvality ktoré sú hodnotené v rámci testovania sú do akej miery je produkt využívaný na cieľený zmysel, overenie zda reálny používatelia zvládajú použiť systém, sledovaním správania používateľov odhaľuje chyby a miesta na zlepšenie. Cieľom je zlepšiť aplikáciu aby vyhovovala jej používateľom.

Pred začiatkom testovania je potreba stanoviť si formu testovania. Forma testovania sa skladá z definície ktoré kupiny budú participovať v teste, čo je zmyslom testovanaia, pre koho je zamerané, aké technológie budú testované, či sa jedné o prototyp alebo výslednú aplikáciu, kde sa bude testoavnie fyzicky vykonávať.[13]

1.8.1 Testovania mladistvým

Testovanie použiteľnosti mladistvými sa odlišuje od bežného testovania nakoľko mentálny model mladistvého je odlišný od dospelej osoby. Tomuto mentálnemu modelu treba prispôsobiť aj následné testovanie.

Aby sa nám podarilo dosiahnúť čo najlepšiu vierohodnosť testovania musí byť dĺžka testovania prispôsobená schopnostiam mladistvého. [**usability_children**]

1.9 Webové inžinierstvo

1.9.1 Synchronizácia dát

Synchronizáciu dát medzi klientami je možné dosiahnť viacerými spôsobmi pričom sa líšia topografickou štruktúrou ktorú využívajú na synchronizáciu ako aj kontrolou nad dátami. Medzi hlavné architektúry ktoré sa využívajú na komunikáciu a synchronizáciu dát patrí klient-server architektúra a peer-to-peer architektúra.

1.9.2 Klient-server architektúra

Klient-server architektúra je sieťový model v ktorom dve hlavné entity, klienti a server, komunikujú medzi sebou za účeľom splenenie špecifickych úloh alebo zdieľania dát. Klient iniciuje požiadavku, čaká na odpoveď serveru a následne zobrazí dáta používateľovi. Server spracúva požiadavky, získa potrebné dáta a pošle ich klientovi.

Výhodou tejto architektúry je centralizovaná správa prostriedkov a dát. Táto výhoda sa zároveň stáva aj nevýhodou v podobe jediného bodu zlyhania. V prípade zlyhania serveru dochádza k nedostupnosti prostriedkov a dát.

Architekúra môže byť rozšírená o ďalšie prvky medzi serverom a klientom v podobe middlewaru.

1.9.3 Peer-to-peer architektúra

Táto architektúra je vyznačovaná absenciou centrálneho prvku ktorý by slúžil ako zdroj pravdy. Komunikácia prebieha priamo medzi klientami a je potreba využiť synchronizačný algoritmus ktorý umožní vzájomnú výmenu dát.

1.9.4 API

Application Programming Interface (API), v slovenskom preklade rozhranie pre programovanie aplikácií, predstavuje súbor pravidiel, protokolov a nástrojov, ktoré umožňujú vzájomnú komunikáciu a výmenu dát medzi odlišnými softvérovými systémami. V modernom softvérovom inžinierstve plní API funkciu sprostredkovateľa (tzv. „middleman“), ktorý definuje presný spôsob, akým môžu klientske aplikácie požadovať a prijímať funkcionalitu alebo dáta od servera.

Z architektonického hľadiska API abstrahuje komplexnosť implementácie backendových systémov. Pre vývojárov to znamená možnosť integrovať externé služby (napr. platobné brány, mapové podklady alebo meteorologické dáta) bez nutnosti vyvíjať tieto riešenia od základu. Tým sa zvyšuje efektivita vývoja, modularita systémov a schopnosť inovácie.

Základné komponenty interakcie cez API:

Klient : Strana, ktorá iniciuje požiadavku.

Server : Systém, ktorý požiadavku spracováva a poskytuje odpoveď.

Koncový bod (Endpoint) : Špecifická URL adresa, na ktorú sú požiadavky smerované.

Metódy : Inštrukcie definujúce typ operácie.

Dátový formát : Štandardizovaná štruktúra prenosu dát, najčastejšie reprezentovaná formátmi JSON alebo XML.

1.9.4.1 Druhy API

Voľba konkrétneho typu API závisí od požiadaviek na stavovosť, výkon a komplexnosť dátových štruktúr. Na základe poskytnutého textu rozlišujeme nasledujúce kľúčové prístupy

REST predstavuje v súčasnosti najrozšírenejšiu architektúru pre webové služby. Jeho kľúčovou vlastnosťou je bezzstavovosť (statelessness). To znamená, že každá požiadavka od klienta na server musí obsahovať všetky informácie potrebné na jej pochopenie a vybavenie (vrátane autentifikácie). Server neuchováva žiadne informácie o kontexte klienta medzi jednotlivými volaniami, čo zjednodušuje architektúru servera a zvyšuje škálovateľnosť systému.

SOAP je protokol zameraný na prístup k objektom a webovým službám prostredníctvom formátu XML. Na rozdiel od RESTu, ktorý je architektonickým štýlom, SOAP je striktne definovaný protokol. Využíva jednoduchú štruktúru slovies (GET, POST, PUT, DELETE) a je nezávislý od prenosového protokolu, hoci sa najčastejšie implementuje nad HTTP. Je typický pre podnikové riešenia vyžadujúce formálnu štruktúru správ.

GraphQL API

gRPC API

1.9.5 OAuth

1.9.6 Cloud computing

Cloud umožňuje zaobstaranie potrebného hardvéru podľa aktuálnych potrieb softvérovej aplikácie.

1.9.6.1 Typy služieb

Podľa úrovne s ktorou pracujeme rozlišujeme viacero úrovní služieb, ktoré nám cloud poskytuje.

IaaS Infraštruktúra ako služba poskytuje používateľom správu nad procesmi, sieťovými prvkami, úložným priestorom a ďalšími základnými výpočtovými prvkami umožňujúc používateľom spustiť hocijaký softvér od celých operačných systémov až po špecifické aplikácie.

V tomto modeli dodávateľ zodpovedá iba za fyzickú infraštruktúru a poskytuje služby pomocou virtualizácie. Používateľ má tak maximálnu kontrolu nad svojím prostredím akoby ho celé vlastnil vo vlastnej réžii.

PaaS Platforma ako služba poskytuje používateľovi predom nakonfigurované prostredie do ktorého môže nasadiť svoju aplikáciu. Toto prostredie zvykne byť definované dostupným programovacím jazykom, nainštalovanými knižnicami a dostupnými nástrojmi.

V tomto modeli dodávateľ zodpovedá a spravuje všetko až na samotnú nasadenú aplikáciu, ktorá zostáva v správe používateľa.

SaaS Softvér ako služba poskytuje priamo prístup k aplikácii pre koncových používateľov.

1.9.7 Backend ako služba

Špeciálnym prípadom softvéru ako služba je keď poskytovaný softvér slúži ako backend pre iný softvér

1.9.8 Webscraping

V rámci metodológie predkladanej práce zohráva kľúčovú úlohu technológia web scraping (často označovaná aj ako web data extraction alebo web harvesting). Ide o ďalšiu významnú webovú technológiu, ktorú využijem v procese zberu a spracovania dát. V ére informačného nadbytku, kedy je podstatná časť relevantných údajov distribuovaná v neštruktúrovanej podobe na internetových portáloch, predstavuje web scraping nevyhnutný nástroj na ich automatizovaný zber. Na rozdiel od aplikačných programových rozhraní (API), ktoré sú často limitované kvótami, spoplatnením alebo vyžadujú striktnú autorizáciu, web scraping umožňuje prístup k verejne dostupným dátam v reálnom čase priamo z prezentačnej vrstvy webu (HTML). Táto technológia tak poskytuje vysokú mieru flexibility a nezávislosti od obmedzení tretích strán.

1.9.8.1 Fázy procesu webscraping

Proces automatizovanej extrakcie dát možno definovať ako simuláciu správania používateľa, ktorá prebieha v troch fundamentálnych fázach:

1. Fáza získavania: V tomto kroku dochádza k nadviazaniu komunikácie so serverom prostredníctvom protokolu HTTP. Softvérový agent odošle požiadavku na cieľovú URL adresu a následne prijme odpoveď vo forme surového HTML dokumentu.

2. Fáza extrakcie: Ide o proces identifikácie a izolácie konkrétnych dátových prvkov v rámci prijatého kódu. Na presnú lokalizáciu údajov sa využívajú technológie ako dopyty XPath, CSS selektory alebo parsovacie knižnice, ktoré dokážu v neštruktúrovanom HTML dokumente nájsť vopred definované informácie.

3. Fáza transformácie: Finálnym krokom je konverzia extrahovaných údajov do štruktúrovaného formátu. Dáta sú následne perzistované v úložiskách typu CSV, SQL databáza alebo JSON, čo umožňuje ich ďalšie analytické spracovanie v rámci praktickej časti práce.

1.9.8.2 Etické otázky ohľadom procesu webscraping

Implementácia automatizovaného zberu dát so sebou prináša špecifické výzvy v oblasti etiky a sieťovej neutrality. Hoci je web scraping vo všeobecnosti akceptovaný pre akademické účely, musí byť realizovaný v súlade s pravidlami vlastníkov sieťových zdrojov. Kľúčovým aspektom je predchádzanie preťaženiu serverov (DoS) prostredníctvom implementácie časových oneskorení medzi jednotlivými požiadavkami.

Z hľadiska transparentnosti a povolenia prístupu je kľúčovým nástrojom súbor robots.txt, definovaný v rámci štandardu Robots Exclusion Standard. Tento mechanizmus vznikol ako konsenzus autorov webových robotov už v roku 1994 s cieľom chrániť servery pred nežiaducim prístupom do citlivých alebo nevhodných častí webu, akými sú dočasné informácie, nekonečné virtuálne stromy či skripty s vedľajšími účinkami. Prevádzkovateľ servera prostredníctvom tohto súboru, umiestneného v koreňovom adresári, definuje prístupovú politiku pomocou polí User-agent (určenie konkrétneho robota) a Disallow (vymedzenie zakázaných ciest). Hoci tento štandard nie je právne vynútiteľný a neexistuje absolútna záruka jeho dodržiavania všetkými agentmi, v akademickej obci a profesionálnej praxi sa považuje za fundamentálny etický kódex. Rešpektovanie týchto inštrukcií je v mojej práci primárnym predpokladom pre zodpovedný a korektný výskum v digitálnom priestore.

Kapitola 2
Analýza

2.1 Prípady použitia

UC-1: **Moje napredovanie**
Aktér: Skaut
Po otvorení aplikácie sa zobrazia úlohy ktoré si používateľ zvolil ako sledované. Zo zoznamu úloh si môže pridať nové úlohy medzi sledované. Po dokončení úlohy sa mu úloha odstráni zo sledovaných.

UC-2: **Vlastná úloha**
Aktér: Skaut, Radca
Skaut zvolí úlohu ktorú chce nahradiť, navrhne nové znenie, úplne nové alebo úprava stávajúceho, a pošle ho radcovi na schválenie. V procese schvaľovania môžu ešte znenie novej úlohy pozmeniť. Keď úloha zodpovedá očakávanej náročnosti, tak ju schváli a skautovi sa pridá medzi aktívne úlohy.

UC-3: **Splnenie úlohy**
Aktér: Skaut, Radca
Po splnení úlohy môže nastať schvaľovanie splnenia zo strany radcu. Radca môže zadať hromadné splnenie úlohy členom svojej družiny.

UC-4: **Moja družina**
Aktér: Radca
Používateľovi sa zobrazí postup všetkých členov jeho družiny.

UC-5: **Pridanie člena do družiny**
Aktér: Radca
Radca môže pridť člena do družiny pomocou jeho mailu, alebo môže vytvoriť QR kód pomocou ktorého sa používateľ po prihlásení automaticky pridá do družiny.

UC-6: **Môj zbor**
Aktér: Zborový
Používateľovi sa zobrazia všetci členovia jeho zboru a môže spravovať ich priradenie do družín. Nastavuje Družinových radcov.

UC-7: **Úprava programu**
Aktér: Programový
Pri zmenách vo vdelávacio výchovnom programe môže programový vedúci zmeniť, pridať alebo odstrániť časti programu. Prípadne môže vytvoriť úplne nový program alebo zrušiť už existujúci.

2.2 Zber požiadaviek

Zber požiadaviek bol realizovaný .

2.2.1 Požiadavky

REQ-1: **Registrácia**
FURPS+: Funkčný
MoSCoW: Musí mať
Aplikácia umožní registráciu nového používateľa.

REQ-2: **Prihlásenie**
FURPS+: Funkčný
MoSCoW: Musí mať
Aplikácia umožní prihlásenie poučívateľa do jeho používateľského účtu.

REQ-3: **Pridanie sa do družiny**
FURPS+: Funkčný
MoSCoW: Musí mať

REQ-4: **Pridanie do družiny pomocou QR kódu**
FURPS+:
MoSCoW:

REQ-5: **Prezeranie programovej ponuky**
FURPS+: Funkčný
MoSCoW: Musí mať

REQ-6: **Pridanie časti programu do wishlistu**
FURPS+: Funkčný
MoSCoW: Musí mať

REQ-7: **Začať program z ponuky/wishlistu**
FURPS+: Funkčný
MoSCoW: Musí mať

REQ-8: **Sledovanie splnených bodov programu**
FURPS+: Funkčný
MoSCoW: Musí mať

REQ-9: **Moja družina - skaut**
FURPS+:
MoSCoW:

REQ-10: **Moja družina - radca**
FURPS+:
MoSCoW:

REQ-11: **Upozornenie na začatie červeného stupňa odborky bez absolvovaného zeleného stupňa**
FURPS+:
MoSCoW:

REQ-12: **Zobrazenie výzvy**
FURPS+:
MoSCoW:

REQ-13: **Poslat progres druziny na mail**
FURPS+:
MoSCoW:

2.3 Cieľová skupina aplikácie

Na základe rozhovorov so zákazníkom a na základe lepšiemu porozumeniu cieľového používateľa aplikácie som dospel ku definovaní nasledujúcich persón podľa časti 1.7.4.

2.4 Potreby backendu

Na základe požiadavkov vyplývajú nasledujúce funkcionality ktoré sú vyžadované od prípadného backendu. Medzi ne patrí potreba autentizácia a autori-

zácie používateľov, synchornizácia dát používateľov cez centrálnu databázu.

2.5 Ne-cloudové riešenie

Klasické umiestnenie backendu na vlastný server vyžaduje vysokú počiatočnú investíciu, takáto investícia je nevhodná pre dobrovoľnícku neziskovú organizáciu. Zároveň by neumožňovala rýchlu odozvu na počet aktívnych používateľov.

2.6 Cloudové riešenie backendu

Umiestnenie backendu pre aplikáciu do cloudu by umožnilo adekvátnejšie využívanie prostriedkov na potreby backendu.

2.6.1 IaaS riešenia

2.6.1.1 AWS

2.6.1.2 OCI

2.6.1.3

2.6.2 PaaS riešenia

2.6.3 BaaS riešenia

2.6.4 FireBase

2.6.4.1

Kapitola 3
Návrh

Táto kapitola sa venuje návrhu frontendu a backendu. Návrh frontendu sa skladá z LoFi a HiFi prototypu. Návrh backandu sa skladá z návrhu API štruktúry.

3.1 Doménový model

3.2 Procesy

3.3 Aktualizácie obsahu programu SLSK

Program SLSK sa síce nemení často, ale môže dôjsť buď k zmene jednotlivých častí, pridanie novej časti napríklad novej výnimočnej výzvy alebo môže dôjsť ku zrušeniu nejakej časti. Na zachovanie možnosti prehliadania si programovej ponuky SLSK musia byť všekty jeho časti uložené aj v zariadení spolu s aplikáciou. Pri štarte aplikácie sa uskutoční dotaz na server ohľadom nových častí programovej ponuky a v prípade zmeny sa tieto zmeny prepíšu do zariadenia používateľa.

3.4 Architektúra aplikácie

3.5 Voľba programovacieho jazyku

3.6 LoFi prototyp

3.7 Testovanie LoFi prototypu

3.8 HiFi prototyp

3.9 Testovanie HiFi prototypu

3.10 API štruktúra

Kapitola 4

Implementácia

4.1 Získavanie dát

Programová ponuka slovenského skautingu sa nachádza primárne na webových stŕankach. Obsahom stránok je detailný popis odboriek, výziev, voľných programových modulov a najvyšších programových ocenení. Na získanie dát ohľadom programu som zvolil webscraping1.9.8 so snahou uložiť tieto dáta do štruktúrovanej podoby, ktorú môžem následne využiť v mojej aplikácii.

Analýzou webových stránok slovenského skautingu som dospel k nasledujúcej stratégií. V prvom kroku som získal postupne všetky odborky, ktoré boli na stránkach rozdelené podľa vekových kategórií. V druhom kroku som sa pustil do výziev ktoré boli náročnejšie v dôsledku laxnejšej štruktúry čomu bolo treba upraviť selektory na extrakciu textu a dátový model do ktorého som ich ukladal. V poslednom kroku som extrahoval z web stránky najvyššie programové ocenenia. Voľné programové moduly neobsahovali takmer žiadnu spoločnú štruktúru naprieč sebou, z tohto dôvodu boli vynechané počas získavania dát.

Na samotnú činnosť procesu webscraping som zvolil knižnicu todo pre programovací jazyk Python. Táto knižnica obsahovala potrebné nástroje na sťahovanie HTML obsahu webstránok a potrebné CSS selektory na extrakciu žiadaného textu.

Získané dáta som ukladal do súborov vo formáte JSON.

4.2 Flutter

4.3 Nasadenie aplikácie

Kapitola 5

Testovanie

5.1 Používateľské testovanie

5.2 Heuristická analýza

5.3 Automatizované testovanie

Kapitola 6

Záver

6.1 Budúci vývoj

Dodatok A
Nějaká příloha

Sem přijde to, co nepatří do hlavní části.

Bibliografia

1. *Výročná správa za rok 2024* [online]. [B.r.]. [cit. 2026-02-10]. Dostupné z : https://skauting.sk/ustredie/publikacie/vyrocne-spravy/vyrocna-sprava-2024/.
2. DIRGOVÁ, Mária. *Skauting: Viac než len zábava v prírode* [online]. 2024. [cit. 2026-03-09]. Dostupné z : https://skauting.sk/skauting/13668/skauting-viac-nez-len-zabava-v-prirode/.
3. *Stupne napredovania* [online]. 2025. [cit. 2026-02-11]. Dostupné z : https://skauting.sk/skauti/program/stupne-napredovania/.
4. MITRÍK, Ján; SUVÁK, Marián. *Prvý skaut: skautský chodník : 1. stupeň.* Bratislava: Slovenský skauting, 2017. ISBN 978-80-89776-12-2.
5. MITRÍK, Ján; SUVÁK, Marián. *Neznáme cesty: skautský chodník : 2. stupeň.* Bratislava: Slovenský skauting, 2019. ISBN 978-80-89776-18-4.
6. MITRÍK, Ján; SUVÁK, Marián. *Posledný vrchol: skautský chodník : 3. stupeň.* Bratislava: Slovenský skauting, 2019. ISBN 978-80-89776-27-6.
7. BEŽILLA, Jakub; SUVÁK, Marián. *Rangerský horizont: príručka pre rangerky a rangerov.* Bratislava: Slovenský skauting, 2019. ISBN 978-80-89776-22-1.
8. *Výzvy* [online]. 2025. [cit. 2026-02-13]. Dostupné z : https://skauting.sk/skauti/program/vyzvy/.
9. *Voľné programové moduly* [online]. 2025. [cit. 2026-02-13]. Dostupné z : https://skauting.sk/skauti/program/volne-programove-moduly/.
10. *Najvyššie programové ocenenia* [online]. 2025. [cit. 2026-02-13]. Dostupné z : https://skauting.sk/skauti/program/najvyssie-programove-ocenenia/.

11. HOSSAIN, Mohammad Ikbal. Software development life cycle (SDLC) methodologies for information systems project management. *International Journal for Multidisciplinary Research*. 2023, roč. 5, č. 5, s. 1–36. Dostupné tiež z: https://pdfs.semanticscholar.org/9eed/fc508509d415c305116ffb258ff5147fd8b8.pdf.

12. SWIERENGA, Sarah J.; PROPST, Dennis B.; ISMIRLE, Jennifer; FIGLAN, Chelsea; COURSARIS, Constantinos K. Mobile Design Usability Guidelines for Outdoor Recreation and Tourism. In: NAH, Fiona Fui-Hoon (ed.). *HCI in Business*. Cham: Springer International Publishing, 2014, s. 371–378. ISBN 978-3-319-07293-7. Dostupné z DOI: 10.1007/978-3-319-07293-7_36.

13. PAVLÍČEK, Josef; PAVLÍČKOVÁ, Petra; PAVLÍČEK, Josef; PAVLÍČKOVÁ, Petra. Usability Testing Methods and Usability Laboratory Management. In: *Updates on Software Usability* [online]. IntechOpen, 2022 [cit. 2026-01-31]. ISBN 978-1-80356-636-8. Dostupné z DOI: 10.5772/intechopen.109140.

Obsah příloh

```
/
├── readme.txt ..... stručný popis obsahu média
├── exe ..... adresář se spustitelnou formou implementace
├── src
│   ├── impl ..... zdrojové kódy implementace
│   └── thesis ..... zdrojová forma práce ve formátu LaTeX
└── text ..... text práce
    └── thesis.pdf ..... text práce ve formátu PDF
```